

# Grouping and Aggregation

Python 101 • Day 6 • summarize tabular data with pandas

**Today's move**

from detail rows



**to summary  
answers**

counts • totals •  
averages

## BLOCK 45

# What this block covers

30 minutes instruction + 30 minutes guided practice

### Group rows

Use `groupby()` to partition a dataframe by category.

### Aggregate values

Calculate size, sum, mean, min, and max by group.

### Build summaries

Reset indexes, sort summaries, and name output columns.

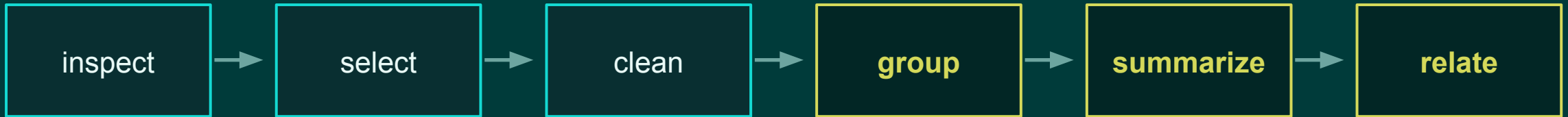
### Verify results

Check grouped totals against the source dataframe.

**Guiding question: What question am I asking about this table?**

## Where we are in Day 6

The morning cleaned rows; the afternoon turns those rows into analytical answers.

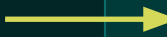


**Grouping is the bridge from cleaned data to decision support.**

## Detail records versus summary information

Aggregation answers questions without showing every row.

| record_id | source    | record_coun | errors |
|-----------|-----------|-------------|--------|
| EQ-1001   | maint     | 12000       | 0      |
| EQ-1002   | logistics | 8500        | 2      |
| EQ-1003   | maint     | 5000        | 1      |
| EQ-1004   | logistics | 9000        | 0      |



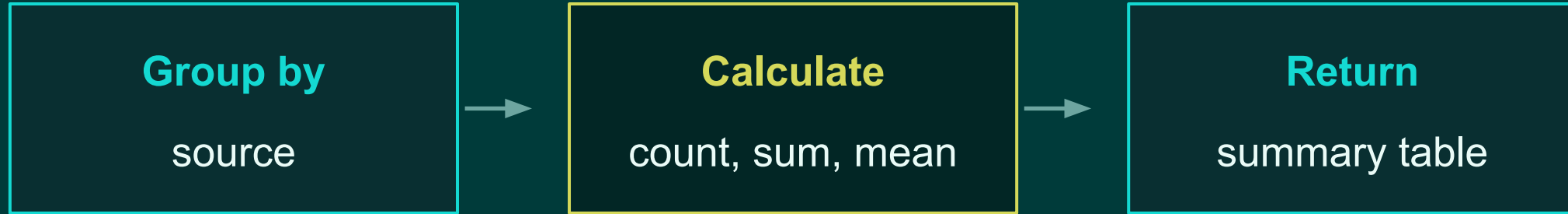
### Summary questions

How many rows per source?  
Which source processed most records?  
Where are errors highest?

**Aggregation compresses many detail rows into a smaller answer table.**

# Aggregation means calculate per group

First decide the category, then decide the calculation.



A grouped result usually has fewer rows and different meaning than the original table.

## Earlier Python: counters and accumulators

Students already know the mechanism under the hood.

```
counts = {}

for record in records:
    source = record["source"]

    if source not in counts:
        counts[source] = 0

    counts[source] += 1
```

### What this loop does

1. Read each row
2. Look at its source
3. Start a counter if needed
4. Add one to that source

**pandas gives a shorter way to express the same data operation.**

## pandas expression: group rows, then count

The code names the operation instead of spelling out the loop.

```
source_counts = (  
    records  
    .groupby("source")  
    .size()  
)
```

### Read aloud

Take the records dataframe,  
partition rows by source,  
then count how many rows  
landed in each group.

The dataframe operation expresses the intent directly.

## groupby() mental model

groupby() defines groups; an aggregation makes the answer visible.

### **maintenance**

EQ-1001  
EQ-1003  
EQ-1008

### **logistics**

EQ-1002  
EQ-1004

### **finance**

EQ-1005



**Aggregation asks each group the same question: count, sum, average, minimum, maximum.**

## BLOCK 45

# Count rows with size()

Use `reset_index()` when you want a normal dataframe output.

```
summary = (  
    records  
    .groupby("source")  
    .size()  
    .reset_index(name="row_count")  
)
```

| source      | row_count |
|-------------|-----------|
| finance     | 25        |
| logistics   | 42        |
| maintenance | 33        |

`reset_index(name=...)` turns a grouped result into a named column.

**size() answers: how many rows are in each group?**

## size() versus count()

Missing values make these two questions different.

| source | record_id | errors |
|--------|-----------|--------|
| maint  | EQ-1      | 0      |
| maint  |           | 1      |
| log    | EQ-3      |        |
| log    | EQ-4      | 0      |

### size()

counts rows  
including missing  
values

### count()

counts non-missing  
values in a column

Use size() for row counts. Use count() when missingness matters.

## Sum values by group

This replaces a loop with a running total for each category.

```
totals = (  
    records  
    .groupby("source")["record_count"]  
    .sum()  
    .reset_index(name="total_record_count")  
)
```

### Interpretation

For each source,  
add all record\_count  
values.



**Ask: should this sum reconcile to the original column total?**

## Mean, minimum, and maximum by group

The same idea works for common statistics.

```
average_errors = (  
    records  
    .groupby("source")["error_count"]  
    .mean()  
)  
  
maximum_errors = (  
    records  
    .groupby("source")["error_count"]  
    .max()  
)
```

### Connection to Day 5

`statistics.mean(values)`

calculates one value.

`pandas mean()` by group  
calculates one value per group.

## One summary table with agg()

Named aggregation creates readable output column names.

```
summary = (
    records
    .groupby("source")
    .agg(
        rows=("record_id", "size"),
        total_records=("record_count", "sum"),
        average_errors=("error_count", "mean"),
        maximum_errors=("error_count", "max"),
    )
    .reset_index()
)
```

### Read it aloud

Create rows by counting.  
Create total\_records by  
summing.

Create average\_errors by  
averaging.

Create maximum\_errors by  
taking max.

**Do not memorize the syntax first. Understand the pattern first.**

## BLOCK 45

# Group by more than one column

Each group is defined by the combination of values.

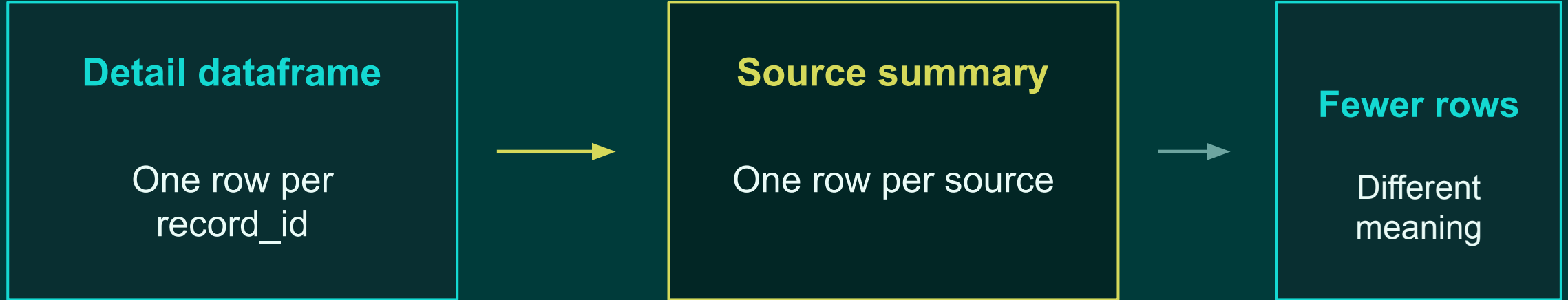
```
source_status = (  
    records  
    .groupby(["source", "status"])  
    .size()  
    .reset_index(name="rows")  
)
```

| source    | status   | rows |
|-----------|----------|------|
| maint     | active   | 18   |
| maint     | inactive | 4    |
| logistics | active   | 21   |
| finance   | active   | 11   |

**Group key = source + status. Changing the key changes the answer.**

# Aggregation changes the grain

Grain means what one row represents.



**Never compare detail rows and summary rows as if they mean the same thing.**

## Sort summaries to answer ranking questions

After summarizing, sort by the metric that matters.

```
summary = summary.sort_values(  
    "total_records",  
    ascending=False  
)
```

| source      | rows | total_records |
|-------------|------|---------------|
| logistics   | 42   | 17500         |
| maintenance | 33   | 17000         |
| finance     | 25   | 9500          |

**Sorting does not change the calculation. It changes how quickly the human can see the answer.**

## Verify aggregated totals

A summary should reconcile to the source when every row belongs to exactly one group.

```
source_total = records["record_count"].sum()

summary_total = (
    summary["total_records"].sum()
)

print(source_total)
print(summary_total)
print(source_total == summary_total)
```

### Verification habit

After a transformation,  
ask whether totals should match.

If yes, prove it with code.

**This is the data version of testing your function output.**

# Common aggregation mistakes

Most errors come from asking a vague question.

## Wrong group key

Grouping by status when the question asks by source.

## Wrong metric

Counting rows when the question asks total records processed.

## Missing reset

Getting a Series when the next step expects a dataframe.

## No verification

Trusting a summary without reconciling totals.

**Before coding, say the question in plain English.**

## Guided exercise: build source summaries

Use the cleaned dataframe from the morning blocks.

### Starting point

```
records = pd.read_csv(
    "data/output/clean_records.csv"
)

print(records.columns)
print(records.shape)
```

### You will produce

- row counts
- record-count totals
- average completion
- maximum error count
- source + status summary
- one sorted summary table

**Timebox: 30 minutes total**

## BLOCK 45

# Exercise 1: count rows by source

Goal: create source and row\_count columns.

```
source_rows = (  
    records  
    .groupby("source")  
    .size()  
    .reset_index(name="row_count")  
)  
  
print(source_rows)
```

## Task

Run the code, then identify which source has the largest number of rows.

**Timebox: 4 minutes**

## Exercise 2: total record count by source

Goal: sum a numeric column inside each source group.

```
source_totals = (  
    records  
    .groupby("source")["record_count"]  
    .sum()  
    .reset_index(name="total_record_count")  
)  
  
print(source_totals)
```

### Task

Compare row count to total record count. Are they answering the same question?

**Timebox: 4 minutes**

## Exercise 3: average completion and max errors

Goal: calculate two different statistics by the same group.

```
average_completion = (  
    records  
    .groupby("source")["completion"]  
    .mean()  
    .reset_index(name="average_completion")  
)  
  
maximum_errors = (  
    records  
    .groupby("source")["error_count"]  
    .max()  
    .reset_index(name="maximum_errors")  
)
```

### Task

Find the source with the highest average completion and the source with the highest max errors.

**Timebox: 6 minutes**

## Exercise 4: group by source and status

Goal: use a two-column grouping key.

```
source_status_rows = (  
    records  
    .groupby(["source", "status"])  
    .size()  
    .reset_index(name="rows")  
)  
  
source_status_rows =  
source_status_rows.sort_values(  
    "rows",  
    ascending=False  
)  
  
print(source_status_rows)
```

### Task

Identify the most common source/status combination.

Timebox: 5 minutes

## Exercise 5: create one summary table

Goal: use `agg()` to combine multiple calculations.

```
source_summary = (  
    records  
    .groupby("source")  
    .agg(  
        rows=("record_id", "size"),  
        total_record_count=("record_count", "sum"),  
        average_completion=("completion", "mean"),  
        average_errors=("error_count", "mean"),  
        maximum_errors=("error_count", "max"),  
    )  
    .reset_index()  
    .sort_values("total_record_count", ascending=False)  
)
```

### Task

Print the table. Then round average columns if needed for readability.

**Timebox: 7 minutes**

## BLOCK 45

# Exercise 6: verify grouped totals

Goal: prove the summary reconciles to the detail table.

```
detail_total = records["record_count"].sum()
summary_total =
source_summary["total_record_count"].sum()

print(detail_total)
print(summary_total)

if detail_total == summary_total:
    print("Totals reconcile")
else:
    print("Check the grouping")
```

## Task

Explain why the totals should match in this dataset.

**Timebox: 4 minutes**

# Applied challenge: answer analytical questions

Use code plus a short written interpretation.

## Questions to answer

1. Which source produced the most rows?
2. Which source processed the most records?
3. Which source has the highest average errors?
4. Which source/status combination is most common?
5. Do grouped totals reconcile?

## Required evidence

For each answer, include:

- the pandas code
- the output table or value
- one sentence explaining the result

**Success: answer the business question, not just run the method.**

## BLOCK 45

# Block 45 wrap-up

You can now turn cleaned detail rows into useful summaries.

### Group

Use `groupby()` to partition rows by category.

### Aggregate

Use `size`, `sum`, `mean`, `min`, `max`, and `agg()`.

### Verify

Check that summary totals reconcile when they should.

**Aggregation changes the grain of the data. Always know what one row represents before and after the summary.**

Next: relate tables using primary keys, foreign keys, and joins.